

Globizhub.com Technical Architecture and Product Blueprint

Prepared for: Globizhub

Purpose: Define a modern, distinctive, and scalable technology-company website and web application for IT products and services.

Executive recommendation

Build Globizhub as a **premium public website plus authenticated client portal and internal operations dashboard**. Use a **modular monolith** first. It keeps development, deployment, and security manageable while leaving a clear path to separate services later.

Globizhub should be more than a brochure website. It should explain the company, generate qualified leads, present products and case studies, support clients after a sale, and give staff one place to manage leads, projects, content, and support.

1. Product scope

Public company platform

The public experience should include Home, About, Services, Products, Industries, Case Studies, Technology Capabilities, Insights, Careers, Contact, Request a Proposal, Book a Consultation, Privacy Policy, Terms, and Cookie Policy.

Service pages can cover software product development, web applications, mobile applications, cloud and DevOps, artificial intelligence and automation, UI/UX, cybersecurity and quality assurance, IT consulting, and dedicated development teams.

Each important service page should explain the customer problem, expected business outcome, delivery approach, relevant technology, evidence or case study, frequently asked questions, and a direct call to action.

Client portal

The portal should provide a dashboard, projects, milestones, activity history, shared files, support tickets, messages, meeting notes, estimates or invoices, notifications, company profile, and client team management.

Internal command center

The staff dashboard should manage the sales pipeline, organisations, contacts, proposals, contracts, projects, milestones, support tickets, content, products, team members, reports, permissions, and audit logs.

The public site and application should share a design system but use different navigation patterns. The public site should focus on storytelling and conversion. The portal should focus on tasks, status, search, and secure data access.

2. Recommended technology stack

Layer	Recommendation	Reason
Frontend	React with TypeScript	Reusable UI and strong type safety
Framework	Next.js for independent cloud deployment; Manus full-stack template when building inside Manus	SEO and server rendering, or rapid integrated delivery
Styling	Tailwind CSS with custom design tokens	Responsive development with consistent branding
Components	Radix UI or shadcn/ui	Accessible, reusable interaction primitives
Animation	Framer Motion or lightweight CSS transitions	Premium motion without excessive client-side code
API	tRPC for the integrated application; REST only where external consumers need it	End-to-end types and fewer contract errors
Backend	Node.js with TypeScript	Shared language across the application
Database	MySQL/TiDB-compatible relational database; PostgreSQL is also suitable for independent hosting	Transactions and clear relationships for business data
ORM	Drizzle ORM	Typed schema and migrations
Cache and jobs	Redis-compatible store when demand requires it	Rate limits, queues, and short-lived cache
Storage	S3-compatible object storage	Images, proposals, project documents, and downloads

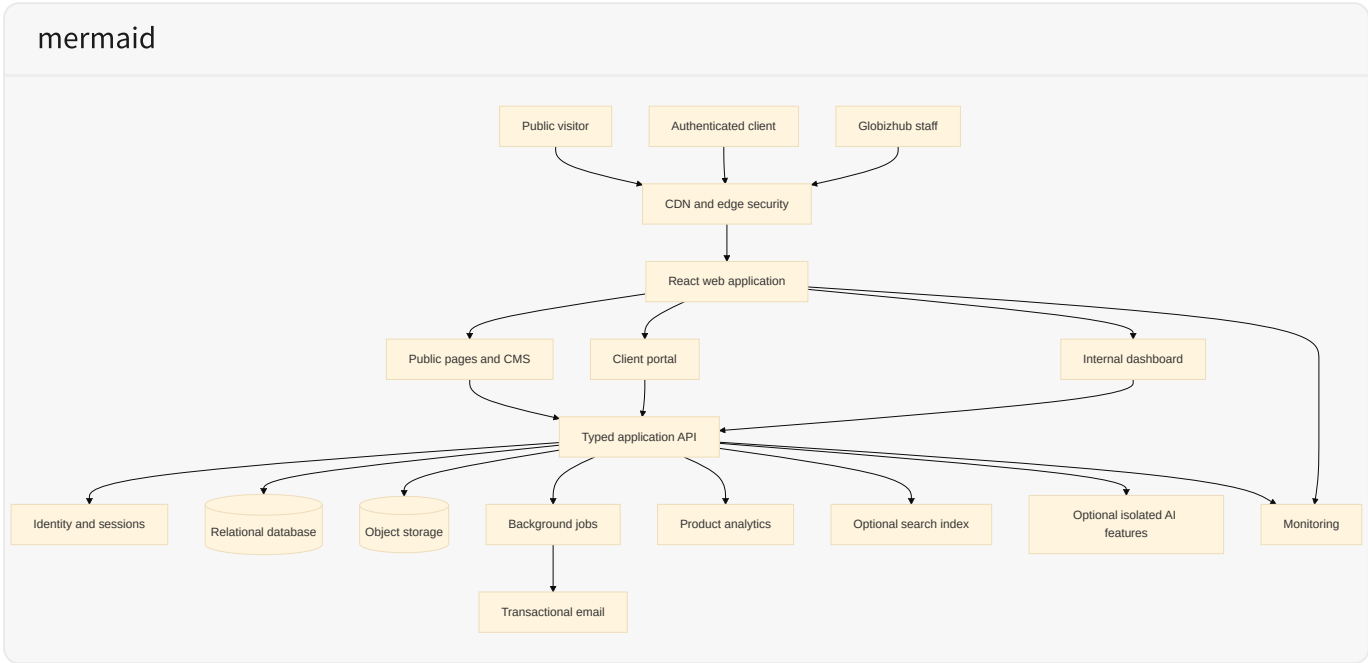
Authentication	Managed OAuth or identity provider with secure session cookies	Avoid building identity security from scratch
Email	Transactional email service with a verified domain	Enquiry confirmations, ticket updates, and notifications
Testing	Vitest, React Testing Library, and Playwright	Unit, component, integration, and browser coverage
Delivery	Git-based CI/CD with preview, staging, and production environments	Safer releases and rollback

Recommended first-release stack inside Manus

Use **React 19, Tailwind 4, Express 4, tRPC 11, Drizzle, a MySQL/TiDB-compatible database, Manus OAuth, and object storage**. This combination already supports typed procedures, authentication, database access, and file storage, which makes it appropriate for an initial Globizhub portal and admin system.

Do not begin with microservices, Kubernetes, or a large event-driven platform. Introduce those only after measured traffic, team size, or product boundaries justify the operational cost.

3. High-level architecture



Architectural style

Use a **modular monolith**. Deploy one application, but divide its code into modules with clear ownership and interfaces:

- Identity and access
- Organisations and contacts
- Leads and CRM pipeline
- Proposals and contracts
- Services and products
- Projects and milestones
- Files and documents
- Support tickets
- Notifications
- Content management
- Reports and audit logs
- Payments, if introduced later

Each module should own its queries and application procedures. Modules should communicate through typed functions or domain events rather than directly modifying another module's tables.

4. Core data model

Entity	Purpose
users	Login identity, profile, status, and activity
roles , permissions	Role-based access control
organisations	Client, prospect, partner, or internal company
organisation_members	Users associated with organisations
leads	Enquiries, source, service interest, and qualification status
lead_activities	Calls, meetings, emails, and notes
services , products	Public service and software catalogue
proposals , contracts	Commercial engagement records

projects	Delivery record, owner, status, and dates
project_milestones	Planned and completed delivery stages
project_members	Internal and client project access
files	Object-storage metadata and permissions
support_tickets , ticket_messages	Support workflow and conversation history
invoices	Optional financial records
content_pages , blog_posts	CMS-managed public content
media_assets	Images and downloadable resources
notifications	In-app and email notifications
audit_logs	Append-only security and operational history

Every business record should have a stable identifier, timestamps, an owner where appropriate, and an archival or soft-delete policy.

5. Main workflows

Lead to project

A visitor submits a structured requirement form. The system validates the data, stores the enquiry and marketing source, sends confirmation, and notifies the assigned sales owner. Staff qualify the lead, schedule a consultation, prepare a proposal, and record the outcome. An accepted proposal can create an organisation, contract, project, and initial milestones.

Client support

A client creates a ticket with category, severity, description, and attachments. The system assigns it by service area, records response deadlines, sends notifications, and displays status changes in the portal. Staff can link a ticket to a project milestone or task.

Content publishing

An editor creates a draft page or article. A reviewer approves it. The publishing system exposes the content with a clean URL, metadata, Open Graph image, structured data, and revision history. The marketing team should be able to update content without changing application code.

File handling

The API should issue short-lived upload permissions. The browser uploads directly to object storage. The API stores metadata and access rules. Files should be private by default and downloaded only through an authorization check and short-lived signed URL.

6. Security baseline

Security belongs in the first release.

- Use HTTPS and secure, HTTP-only, same-site session cookies.
- Enforce role-based access control in the API, not only in the interface.
- Separate public, client, staff, manager, and administrator permissions.
- Validate input with schemas such as Zod.
- Rate-limit login, contact, proposal, upload, and ticket endpoints.
- Reduce spam with throttling, honeypots, and a managed challenge when necessary.
- Restrict uploaded files by size, MIME type, extension, and malware policy.
- Keep secrets in environment configuration and out of source control.
- Record administrative actions, permission changes, authentication events, and sensitive document access.
- Back up the database and test restoration regularly.
- Define retention rules for leads, applicants, tickets, and inactive accounts.
- Publish privacy, cookie, terms, and accessibility information appropriate to the countries where Globizhub operates.

Review security before enabling client documents, invoices, or payment functionality.

7. Performance, SEO, and accessibility

The public site is a sales asset. Render important content server-side or statically where practical. Optimize images, limit third-party scripts, and avoid sending large JavaScript bundles to mobile visitors.

SEO foundations should include semantic headings, descriptive titles, canonical URLs, XML sitemap, robots rules, Open Graph metadata, organisation and article structured data, internal linking, and a clear content taxonomy.

Target WCAG 2.2 AA principles. Provide adequate contrast, visible focus states, keyboard navigation, meaningful labels, alt text, reduced-motion behavior, correct heading order,

and accessible form errors. Animation should enhance the experience but never carry essential meaning by itself.

Useful performance targets are Largest Contentful Paint below 2.5 seconds, Cumulative Layout Shift below 0.1, and Interaction to Next Paint below 200 milliseconds on representative devices and networks. Treat these as continuously monitored targets.

8. Visual direction

Globizhub should feel premium, intelligent, and technically credible rather than like a generic agency template. A strong initial direction would use a graphite foundation, one bright accent such as electric cyan or violet, a warm neutral editorial surface, legible body typography, and a distinctive display face for major headings.

Technical motifs such as grids, nodes, system diagrams, product screenshots, and metric cards can create a recognisable identity. Use them with restraint. The first screen should clearly explain what Globizhub builds, who it serves, why it is credible, and what the visitor should do next.

Create a design system before building many pages. Define color, typography, spacing, radius, shadow, focus, breakpoint, and motion tokens. Keep interaction transitions short and respect `prefers-reduced-motion`.

9. Delivery roadmap

Phase 0: discovery and brand foundation

Confirm target customers, markets, services, brand identity, legal requirements, content owners, and business goals. Produce the sitemap, page inventory, roles, initial data model, and visual direction.

Phase 1: public launch platform

Build the home page, service pages, products, about, industries, case studies, insights, careers, contact, proposal form, CMS, SEO foundation, analytics, and legal pages.

Phase 2: internal CRM and operations

Add staff authentication, lead pipeline, contacts, enquiry assignment, activity notes, proposal tracking, content workflow, file management, audit logs, and reports.

Phase 3: client portal

Add client invitations, organisation membership, project dashboards, milestones, documents, ticketing, messages, notifications, and service-level reporting.

Phase 4: software-product capabilities

If Globizhub launches its own SaaS products, add onboarding, licensing or subscriptions, billing integration, usage analytics, feature flags, API documentation, and a support knowledge base.

Phase 5: optimization and scale

Introduce dedicated search, Redis workers, advanced analytics, service extraction, stronger observability, and multi-region deployment only when measured demand requires them.

10. Engineering standards

Use short-lived feature branches or trunk-based development with code review. Every release should pass formatting, type checking, unit tests, integration tests, end-to-end smoke tests, accessibility checks, and a production build.

Maintain local, staging, and production environments. Automated deployment should create previews, run migrations in a controlled step, and preserve the previous release for rollback. Database changes should remain backward-compatible during deployments where old and new application versions overlap.

The repository should contain environment-variable documentation, architecture decision records, API conventions, coding standards, incident procedures, and a data-retention policy.

11. Definition of a successful first release

A visitor should be able to understand Globizhub's offer, find a relevant service, see evidence of capability, submit a qualified requirement, and receive a reliable confirmation. Staff should manage the enquiry without spreadsheets. An approved client should eventually sign in, view project progress, exchange documents securely, and raise a support request.

Measure qualified enquiries, proposal completion, consultation bookings, service-page engagement, response time, proposal conversion, portal activation, ticket response time, page performance, and organic search growth.

Final recommendation

Start with:

React + TypeScript + Tailwind CSS + accessible components + typed API procedures + Drizzle ORM + MySQL/TiDB-compatible database + managed authentication + S3-compatible storage + transactional email + automated tests and CI/CD.

Add AI later for content search, lead triage, proposal drafting, or support knowledge retrieval. AI must remain isolated behind the same permission system and must not make unreviewed commitments to clients.

This architecture gives Globizhub a distinctive public presence, a practical sales and delivery system, and a safe path toward proprietary software products.

References

- [1] [React documentation](#)
- [2] [TypeScript documentation](#)
- [3] [Next.js documentation](#)
- [4] [Tailwind CSS documentation](#)
- [5] [Web Content Accessibility Guidelines 2.2](#)
- [6] [OWASP Application Security Verification Standard](#)
- [7] [Drizzle ORM documentation](#)
- [8] [MySQL official site](#)
- [9] [W3C accessibility guidance](#)
- [10] [Web Vitals guidance](#)
- [11] [Schema.org structured data vocabulary](#)
- [12] [Playwright documentation](#)
- [13] [Vitest documentation](#)
- [14] [tRPC documentation](#)
- [15] [shadcn/ui documentation](#)
- [16] [Redis documentation](#)
- [17] [Amazon S3 user guide](#)
- [18] [The Twelve-Factor App](#)
- [19] [Monolith First architectural guidance](#)
- [20] [OWASP API Security Project](#)
- [21] [Google Search SEO fundamentals](#)
- [22] [Google structured data introduction](#)
- [23] [OpenTelemetry documentation](#)
- [24] [MDN Content Security Policy documentation](#)
- [25] [MDN HTTP cookies documentation](#)
- [26] [MDN Strict Transport Security documentation](#)

- [27] MDN web security documentation
- [28] MDN web accessibility documentation
- [29] MDN web performance documentation
- [30] Drizzle ORM repository
- [31] Express documentation
- [32] Node.js documentation
- [33] W3C Trace Context specification
- [34] Google SRE monitoring guidance
- [35] NIST Cybersecurity Framework
- [36] OWASP Cheat Sheet Series
- [37] MDN prefers-reduced-motion documentation
- [38] MDN HTML form documentation
- [39] W3C Content Security Policy Level 3
- [40] Redis repository
- [41] MDN Cache-Control documentation
- [42] MDN File API documentation
- [43] MDN Performance API documentation
- [44] MDN Web Authentication API documentation
- [45] OAuth 2.0 resources
- [46] W3C Secure Contexts specification
- [47] MDN Set-Cookie documentation
- [48] WAI-ARIA specification
- [49] MDN Referrer Policy documentation
- [50] Playwright repository

Document author: Manus AI

Note: Technology and provider choices should be revalidated during implementation against Globizhub' s budget, team skills, geography, compliance requirements, and expected traffic.